Toronto Metropolitan University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Programs: Electrical and Computer Engineering

Course Number:	COE 758
Course Title:	Digital Systems Engineering
Semester/Year (e.g.F2016)	Fall 2025
Instructor	Lev Kirischian

Lab Report No.	1
Report Title	Memory Hierarchy: Cache Controller

Section No.	10
Group No.	-
Submission Date:	Oct 29, 2025
Due Date:	Oct 29, 2025

Name	Student Number	Signature*
Ahmad Hafian	*****4283	A.H.
Hasib Bhuiyan	******9402	H.B.

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf

1. Abstract

The objective of the lab was to learn how the cache controller functions and how it interacts with its block memory and the SDRAM controller. The cache controller expects to receive commands and information from the CPU and it will have to decide what to do with these commands and how to interact with the memory stored in it, in the cache itself, and how to communicate with the SDRAM controller as well. The other objective was to design and implement the logic of the controllers and how they interact with the SRAM memory and other logic devices. The design organization of the SRAM controller, SRAM memory blocks, SDRAM controller, and SDRAM memory blocks as well, in addition to dealing with the CPU commands. The final objective was to learn VHDL code and how to use it in the Xilinx ISE CAD environment and implement that and evaluate the VHDL code written using an FPGA microcontroller.

The project began with providing us with CPU-Gen VHDL code, and with providing the requirements and specifications of the organization and design of the SRAM, SDRAM, and their controllers, and building on these specifications a VHDL code that fulfills these requirements and specifications. For the memory block of the SRAM, a tutorial was provided on how to implement an SRAM unit inside the CAD program. And we will have to write a VHDL code that allows us to use that memory block and transfer its content to the CPU. The objectives of the project were achieved by requiring us to write the VHDL code of the cache controller unit. Building upon Tutorial 1 through Tutorial 3, and adding other requirements, we achieved the objectives of the project. The requirements that achieved the objectives directly were writing the VHDL code for the cache controller and testing it on a specific board that is supported by Xilinx ISE CAD. Also behavioral cases of the SRAM controller were given and we had to implement them using the VHDL code.

After implementation, the results were promising, which were a Chipscope waveform that changed between five states between five to achieve the four behavioral cases specified in the project manual. In addition, an SD-RAM controller that is simple while fulfilling the CPU requirements via the cache controller.

2. Introduction

The objective and the motivation behind this project and the idea of having a cache controller that takes commands from the CPU is utilizing limited resources. The limited resource is the SRAM memory block units available. Increasing the capacity of the SRAM would prove expensive to the design of the chip. So a less expensive option is the SDRAM in order to store program files and instructions that are necessary for the CPU. The SRAM and its controller is an intermediate layer that predicts which program instructions are going to be required in the future and it stores them in its memory block that is faster than the SDRAM memory blocks. This minimizes the amount of time fetching the SDRAM memory blocks for the required program instructions while keeping the cost within a reasonable limit.

3. Specifications

Each device we were required to implement had specific requirements to it and specifications. For example, the SRAM block memory was 8 blocks. Each block had 32 words of 8 bits per word setup and organization, while the SDRAM memory did not have any specific requirements to it. On the other hand, the cache controller has very specific requirements and behavior that is expected from it. A behavior that is expected from it is to receive the address sent from the CPU of the data required and decide whether this address is loaded in one of its blocks. If the data in the address specified exists, then the operation should be labeled as a hit, and the data in the specified block would be sent to the CPU. If the address and its data are not in the SRAM memory block, then there are two cases for the consequent actions. The first case is if the dirty bit is 1, or the second case if the dirty bit is 0. If the dirty bit was 0, a read behavior would be expected from the cache controller, where the cache controller fetches the data in the SDRAM memory block and loads it onto its own block, and then sends it to the CPU. The second case where the D bit is 1, this implies that the data on one of its blocks has been altered, and it needs to be saved to the SDRAM memory before it loads the block that is required and sends one of its works to the CPU.

Another factor that needs to be taken into consideration is the valid bit. It is a bit that decides if the data on the cache memory blocks is an actual data or just a random unimportant data. There is a decision process that happens before hit or miss. If the valid bit is set to zero for a memory block in the cache, then the entire block is considered unusable data and a load operation needs to take place from the SD RAM memory before the data can be read or sent to the CPU.

4. Device design (description)

4a. Symbol

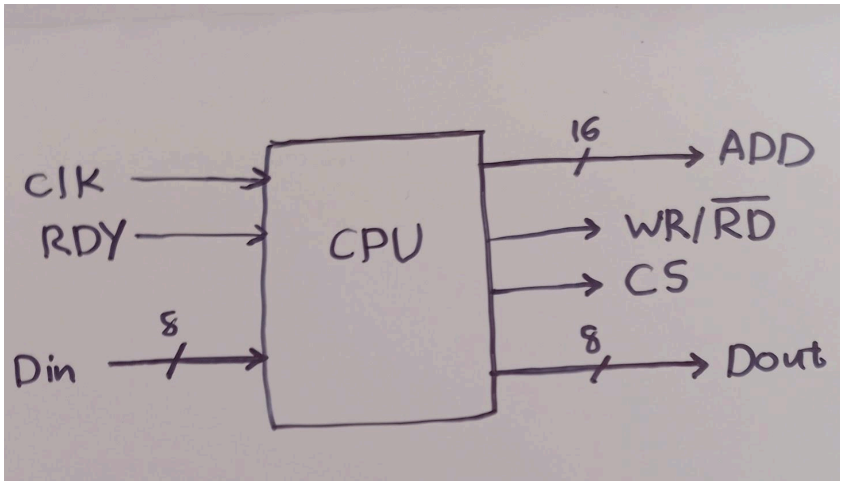

Figure 1: CPU symbol

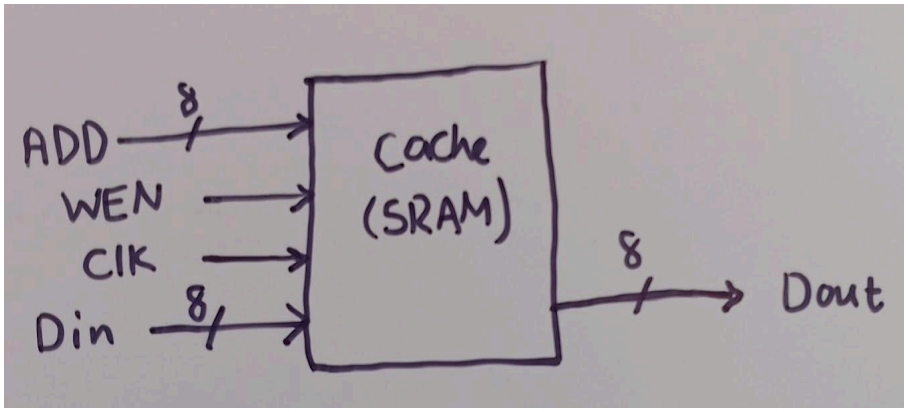

Figure 2: SRAM symbol

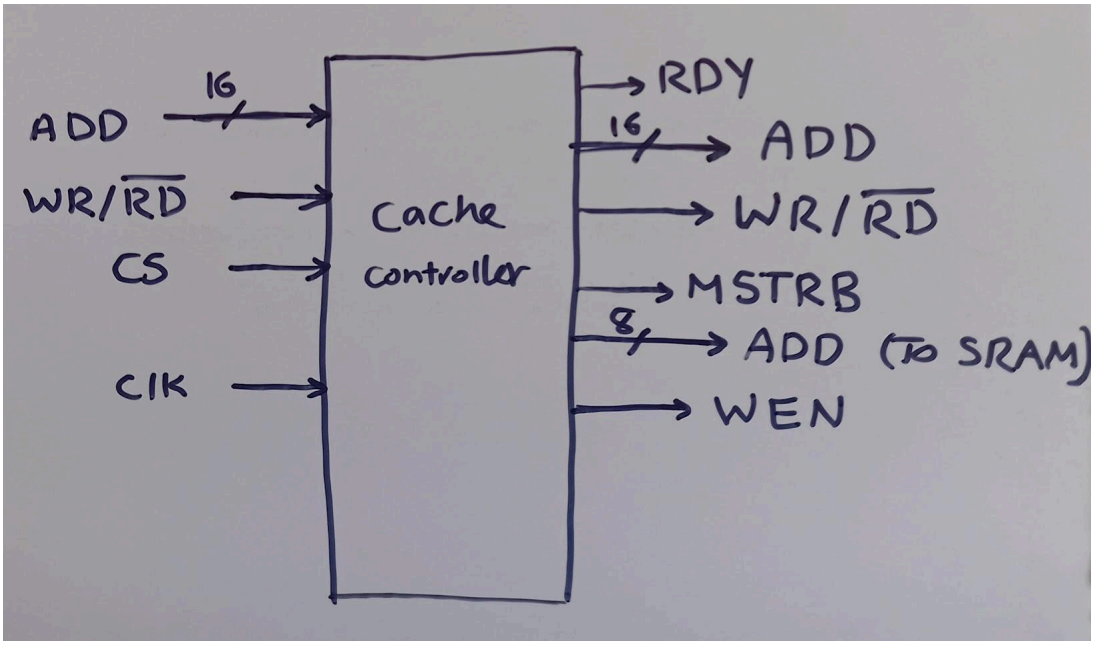

Figure 3: cache controller symbol

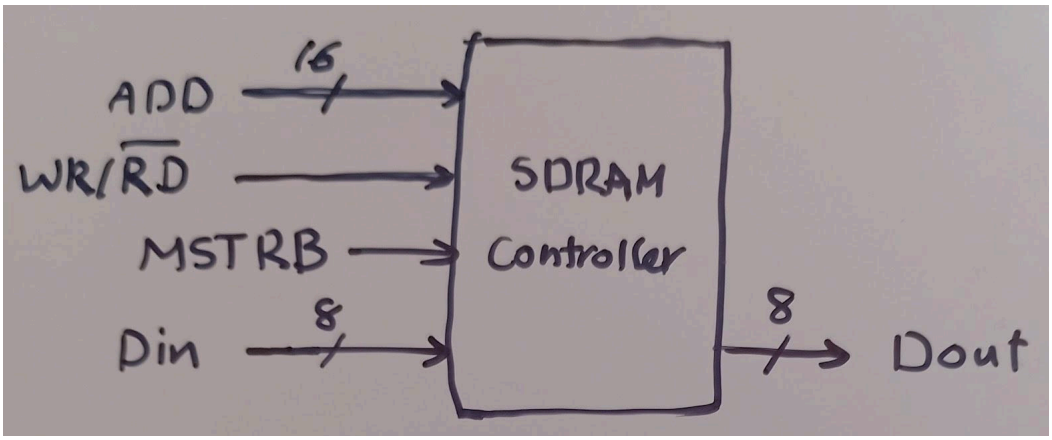

Figure 4: SDRAM Controller Symbol

4b. Block diagram

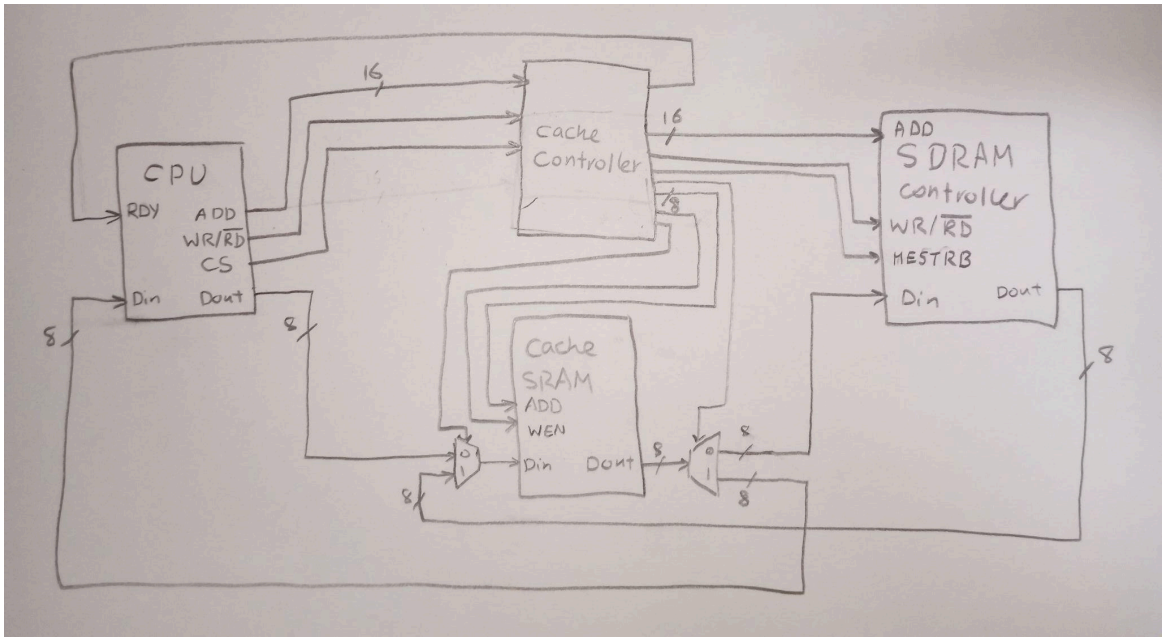

Figure 5: Block diagram

4c. State Diagram:

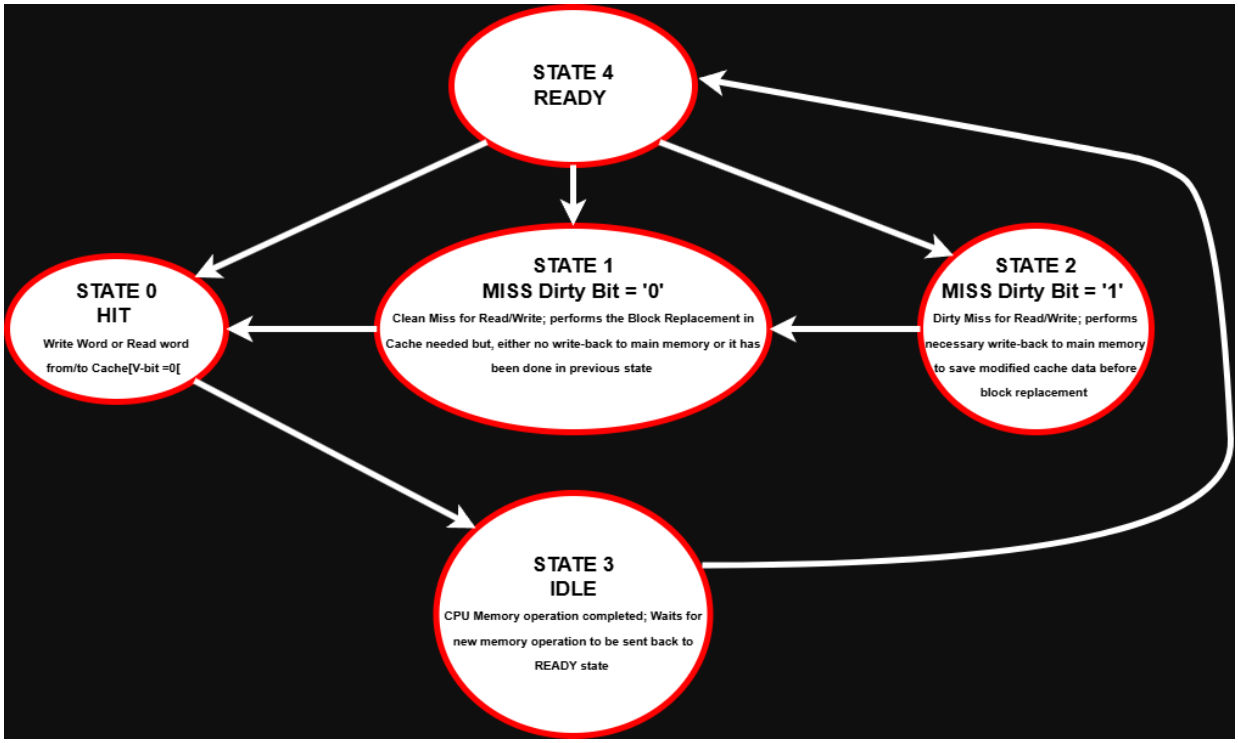

Figure 6: State Diagram

4d. Process Diagram:

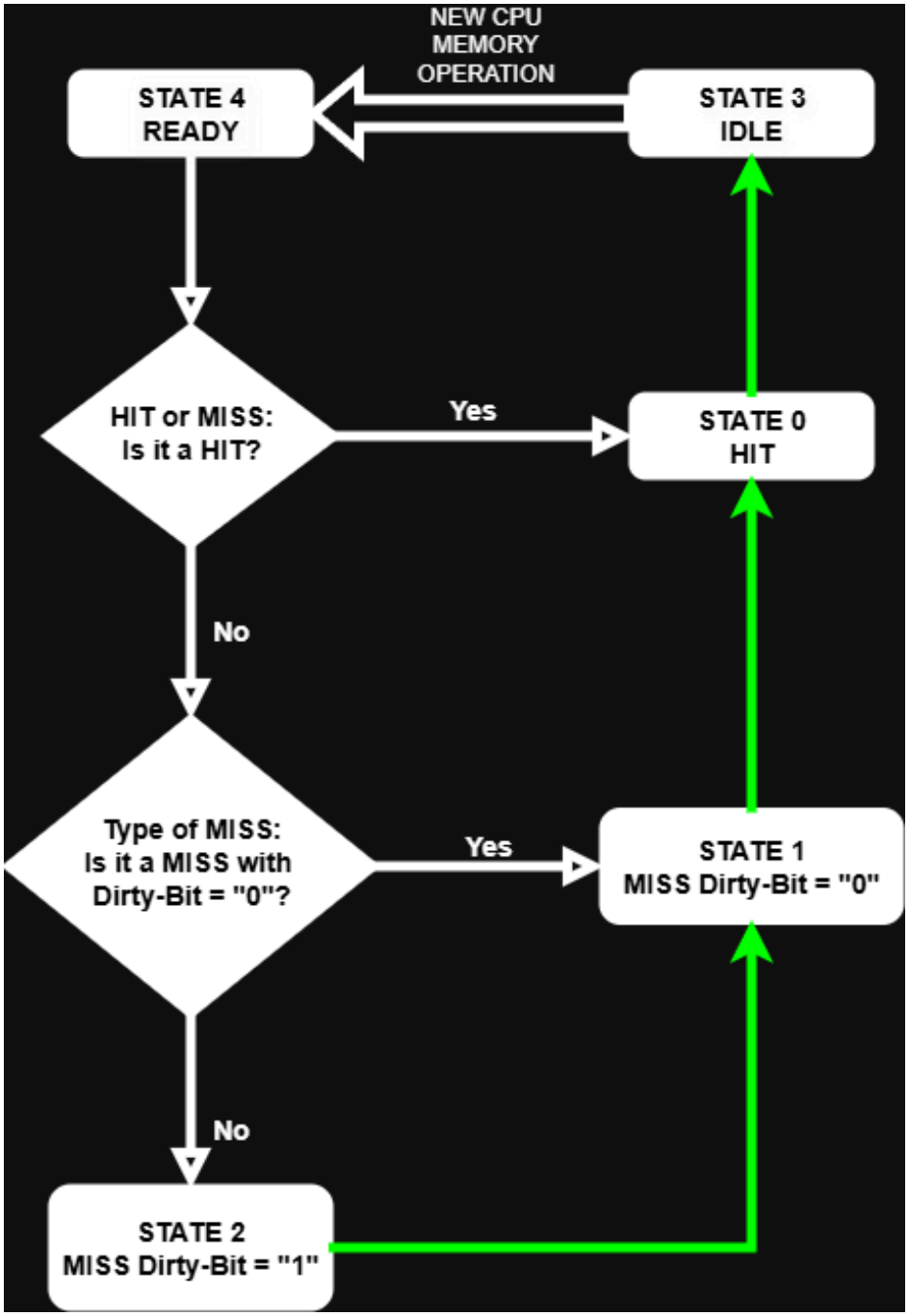

Figure 7: Process diagram

The State Diagram in Figure 6 shows the state flow for the memory operations that are done through the cache controller. As it could be seen there are 5 concrete states in this cache controller, each having its specific purpose.

The first state in which all CPU memory operations must start from is State 4(READY state). Here the memory instruction is loaded and the information regarding the CPUs requested memory task is assigned to the respective signals, as well as given to the cache(through the cache controller), and SDRAM(main memory). On top of all that, during this state the cache controller determines whether the memory instruction is Hit or Miss. According to the cache controller behavioural cases, the cache controller can categorize memory operations within 4 types; Writing a word to cache(Hit-Write), Reading a word form cache(Hit-Read), Read/Write from/to cache where word is not in cache(Miss - Clean Miss, Dirty Bit = "0"), Read/Write from/to cache where word is not in cache and cache block is not written back to main memory after modifications(Miss-Dirty Miss, Dirty Bit = "1"). After the cache controller determines which type of memory operation it is, it passes onto the next state(HIT[Read or Write], Clean MISS, or Dirty MISS), which deals with the corresponding memory operation type.

Do note that these states are not independent of each other, rather they are very much dependent on one another, specifically the clean miss and dirty miss. When the state 'dirty miss' is called, it simply performs its specific task & purpose, which is to back-up the recently modified and unsaved cache block by writing it back to the main memory. It then passes control to the 'clean miss' state, which can then perform block replacement for the new requested memory block for CPU's memory instruction, and following the aftermath it passes onto the next state which is 'hit', and the actual memory instruction(Read/Write) is performed on the newly brought cache memory block. It must be understood that 'dirty miss' operation type is really just a combination of main memory write-back with the clean miss and hit operations. Similarly, the clean miss operations involve block replacement, and the 'hit' operation. These operations/states are much more closer and related than one can expect, they all share common actions(like 'hit') and can simply do their respective specialized task and pass onto the next state; this ultimately makes it efficient by allowing the reusing of code for modular design rather than writing the same common code(etc. like writing the 'hit' code for all operation types) for each of the types.

Now after all these states, when the memory transaction is finally done, the code can go to an IDLE state. This state is simply the variable CPU_READY, indicating that it is done and waiting for the next memory instruction to come, it then goes all over again starting from the READY state.

5. Results

Now, to analyze the resulting timing diagrams and process simulation results, we will be using Isim (functional simulations) within Xilinx ISE, and for hardware simulation the chipscope analyzer will be used. Using these tools and with the help of the internal controller(ICON) and the internal logic analyzer(ILA) we can do sufficient analysis and produce resulting functional and timing diagrams.

Now as previously said, the states & types of behavioral cases are very much dependent on one another. Typically for a ‘dirty miss’ case, it will write-back cache data to main-memory and then proceed to the ‘clean miss’ state where block replacement will occur, finally it will proceed to the ‘hit’ state where CPU desired instruction will be able execute on the appropriate memory block. This implies that by analyzing a ‘dirty miss’ case on our functional and timing simulations, we will be able to record the timings for dirty miss, clean miss, and hit all three will be combined to make the ‘dirty miss’ operation:

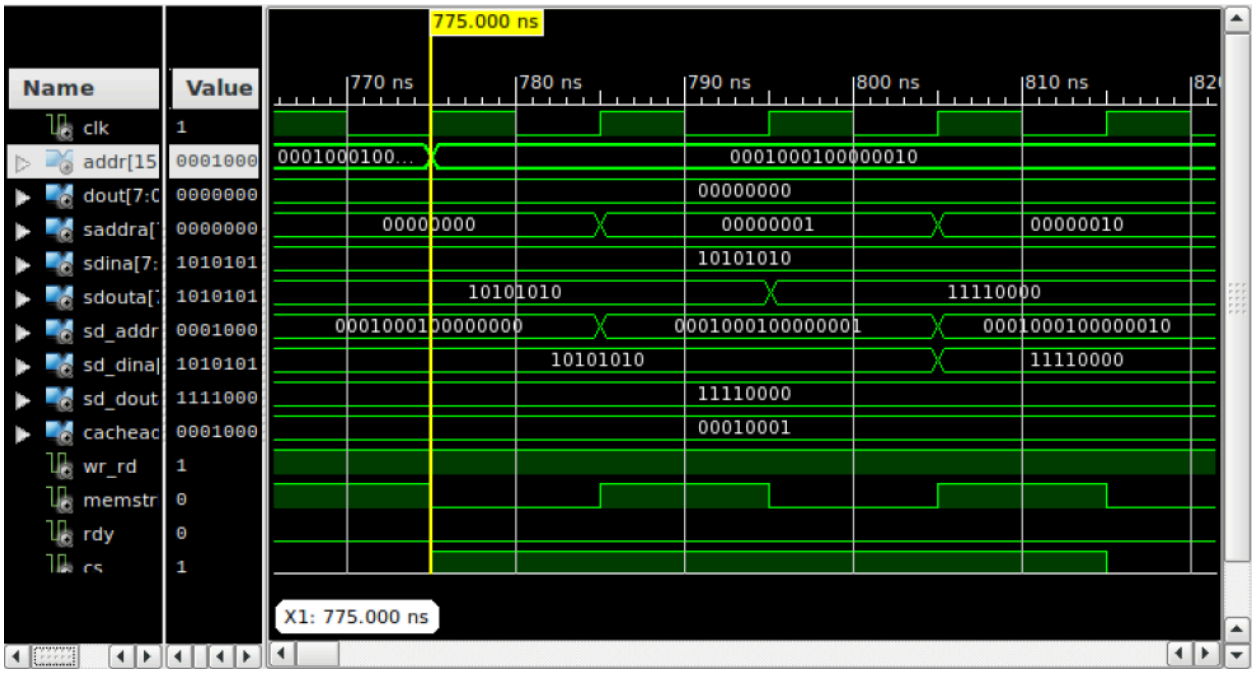

Figure 8: ISIM Testbench Waveform for the start of a “Dirty Miss” transfer

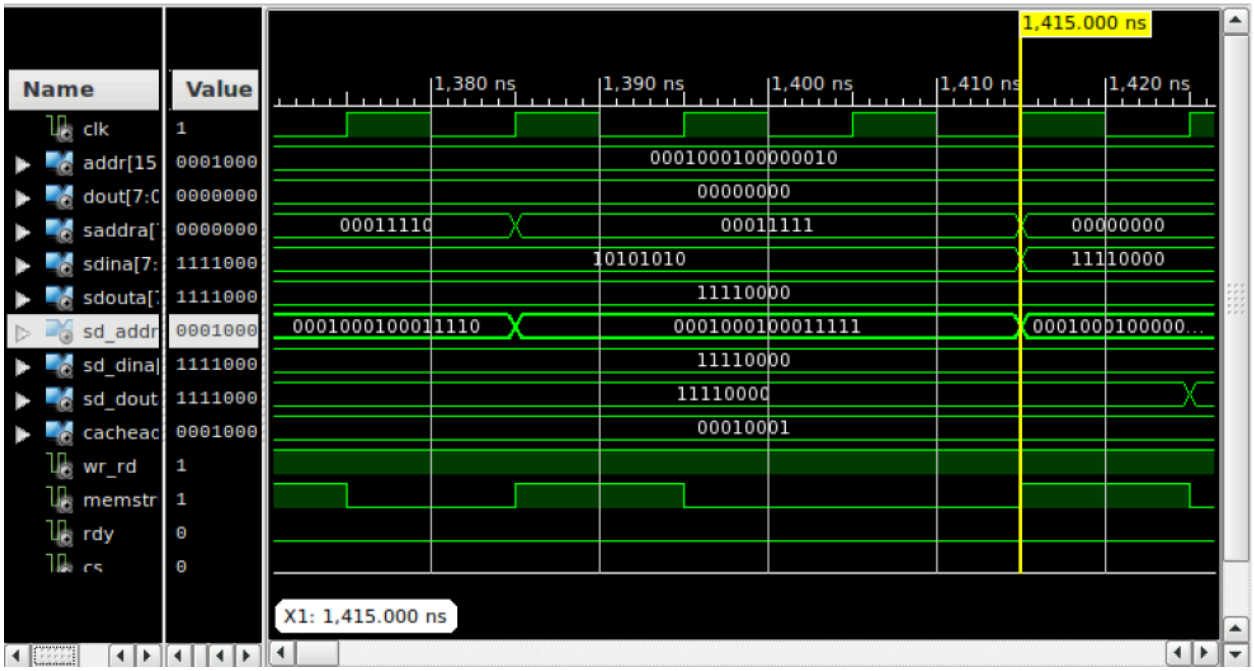

Figure 9: ISIM Testbench Waveform for the end of a “Dirty Miss” transfer

$$\Rightarrow Time\ Delay\ for\ 'Dirty\ Miss'\colon\ 1,415,000\ -\ 775,000\ =\ 640,000\,nS$$

Here we could see the ‘dirty miss’ (state 2) running from beginning to the end which is just before it proceeds to ‘clean miss’(state 1). As it could be seen from both the code and simulation which both complement each other, the ‘dirty miss’ involves 64 clock cycles indicated by the counter in the code. For every even counter, that is every other clock cycle, we can see the SDRAM strobe(SDRAM_MSTRB) is enabled and the data transfer procedure between the main memory and the cache is processed. Note that this is equivalent to 32 cycles as only half of them are used, this is required as data transferring takes some time for signals to be asserted between the SRAM and SDRAM controllers and then for the actual transfer itself. In the span of these 32 cycles, each cycle transfers a 32-bit word from one specific block of the cache to transfer it to the main memory(SDRAM); this means that by the end of the transfer all 32 words of a single specific block will be written back to the main memory, this is the goal of the ‘dirty miss’ state. Now in the waveform/simulation, we could see that the specified address is “0001000100000010”, where bit 15-8 is the TAG, 7-5 is the INDEX, and 4-0 is the offset. In the first image we could see that the new address is being loaded with the SRAM and main memory address changing to the corresponding TAG and Index value. Note that during transfer the offset bits(bits 4-0) are set to 0, and increase by 1 till it reaches 32(‘11111’ on 2nd image) for both addresses; this shows that for the specific TAG and Index that has been found in the main memory, all 32 words within that index block will be transferred to the cache with the same word order(same offset value) and in the same index value(same block number). It could be seen that “sdouta”(SRAM output) and “sd_dina”(DRAM input)exchange values, with the SRAM output

giving output values first and this being directed to the main memory input so cache values can be written back to main memory. The wr_rd value is also set to 1, indicating that this memory operation is write instruction from the CPU.

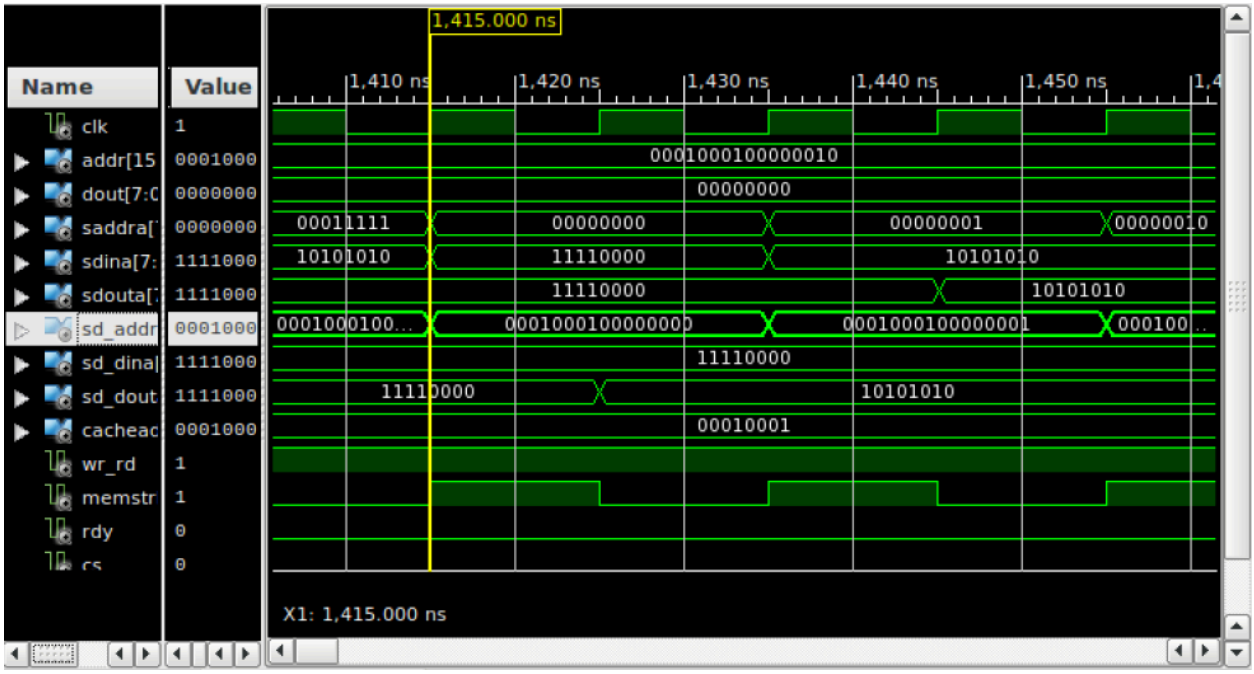

Figure 10: ISIM Testbench Waveform for the start of a “Clean Miss” Transfer

$$\Rightarrow Time\ Delay\ for\ 'CLean\ Miss'\colon\ 2,065,000\ -\ 1,415,000\ =\ 650,000\,nS$$

Now from the ‘dirty miss’ state we proceed to the next which is the ‘clean miss’(state 1). Here we could see very similar works to the previous state’s data transfers, it’s essentially the same, except that instead of the cache reading values to the main memory to be written, it's the other way around with the main memory reading(DRAM output) data to the cache(SRAM input). It could be seen that “sd_dout”(DRAM output) gives out DRAM data values first, and then those same values show up on the “sdina”(SRAM input), indicating that main memory is giving the desired memory block that the CPU needs to the cache which overwrites it in its respective index(as index has been previously written back during last state, we don’t need to worry about what data that was previously inside that index). Note that the remaining variables like “sd_dina” and “sdouta” show the same values that were used in state 2, they are currently inactive and not being used. Now, for this state, again the exchanges are done by 32 transfers, each transfer requiring 2 cycles and 1 word(32-bit), until the entire block from the main memory is written to the cache. This is what the block replacement term means, to replace an entire cache block with the main memory block desired by the CPU.

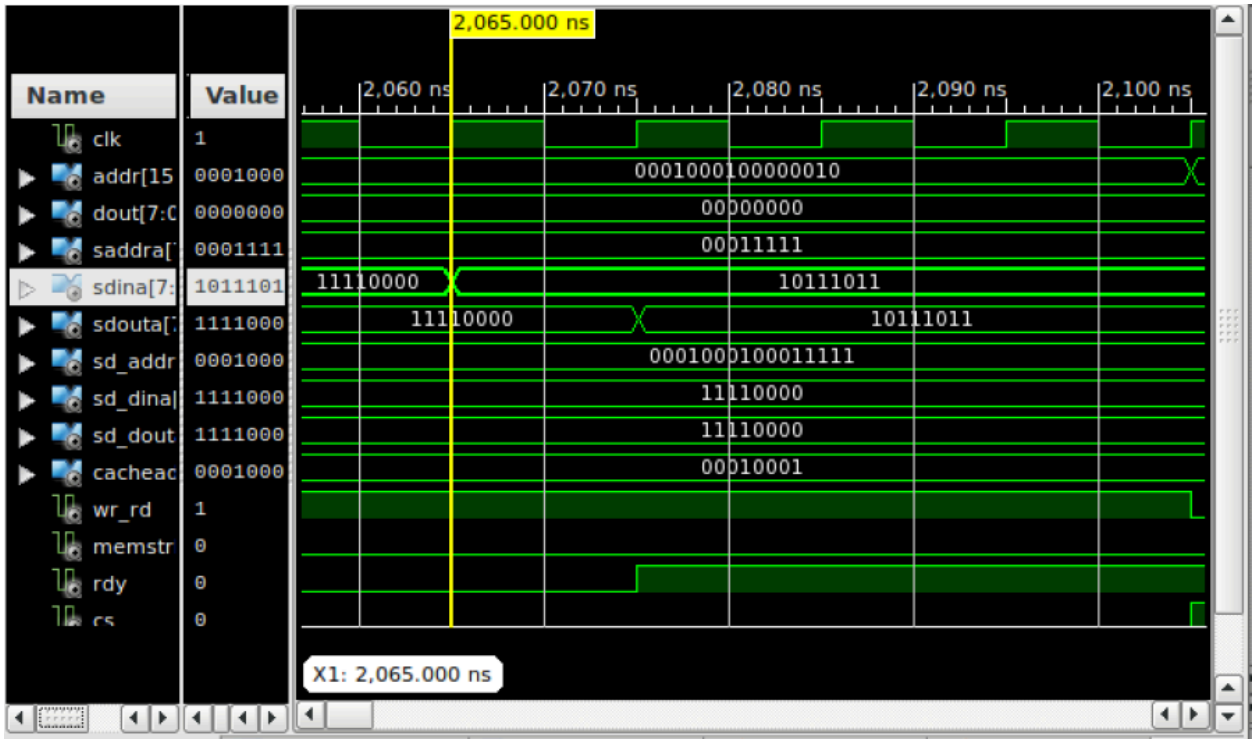

Figure 11: ISIM Testbench Waveform for the start of a “Hit” state

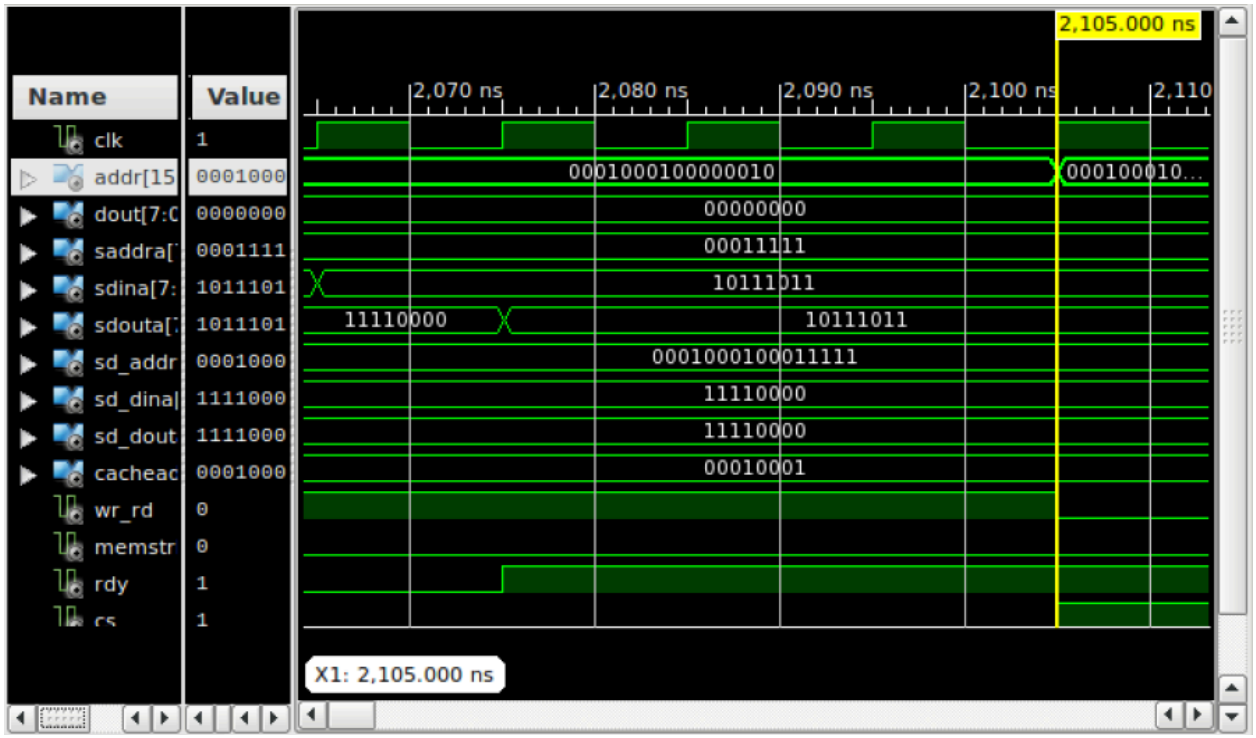

Figure 12: ISIM Testbench Waveform for the end of a “Hit” state and “Idle” state

$\Rightarrow Time\ Delay\ for\ 'Hit'\colon\ 2,105,000\ -\ 2,065,000\ =\ \ 40,000\ nS$

Now the last state that will be passed to is the ‘HIT” state. This is where the CPU finally has the block required, imported from the main memory. Now direct changes(read/write) can be made by the CPU to the cache controller & cache. This state is the shortest, as it simply performs its read or write options and everything is set in the cache. As it could be seen, the wr_rd is 1 in the first image, indicating that the CPU wants to write to the new imported cache block. What does it want to write? Well that is defined in the output from the CPU, which is directed to the “sdina”(SRAM Input). That is why a random, unexpected value appears at the SRAM input, when in reality this is the CPU output data that the CPU wants to be written to the cache block.

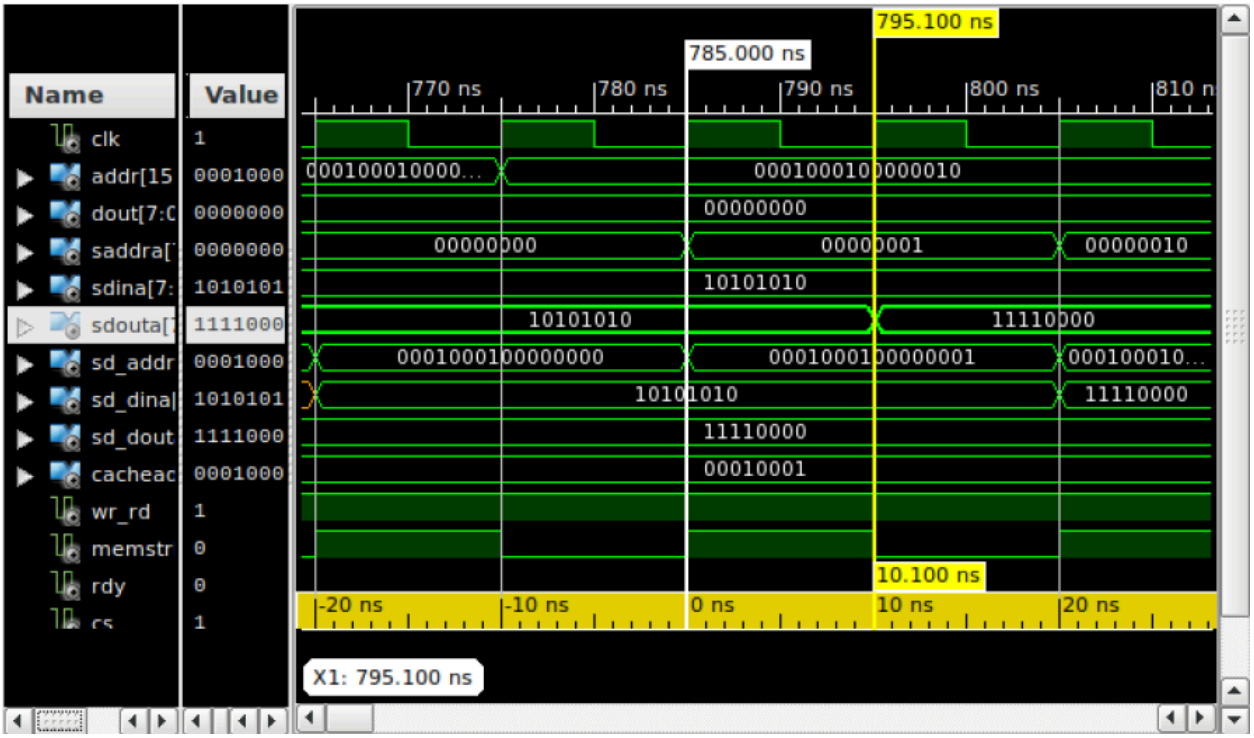

Figure 13: ISIM Testbench Waveform for the delay involved in the Data Access Time

N	Cache Performance Parameter	Time in nS
1	Hit/Miss Determination Time	**60 nS** (Start of State 4 to end of State 4)
2	Data Access Time	**10 nS**
3	Block Replacement Time	‘Clean Miss’ = **650,000 nS**
4	Hit Time(Case 1 and 2)	**40,000 nS**
5	Miss Penalty for Case 3(When D-bit = 0)	‘Hit’ + ‘Clean Miss’ = 40,000+650,000 =

		690,000 nS
6	Miss Penalty for Case 4(when D-bit = 1)	‘Hit’ + ‘Clean Miss’ + ‘Dirty Miss’ = 40,000+650,000+640,000 = **1,330,000 nS**

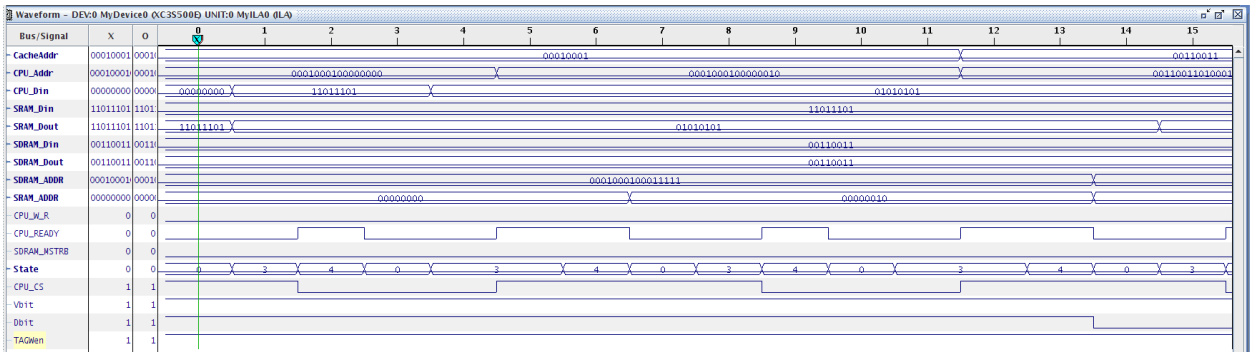

Figure 14: cache controller going through the states: Ready, Hit, and IDLE repeatedly

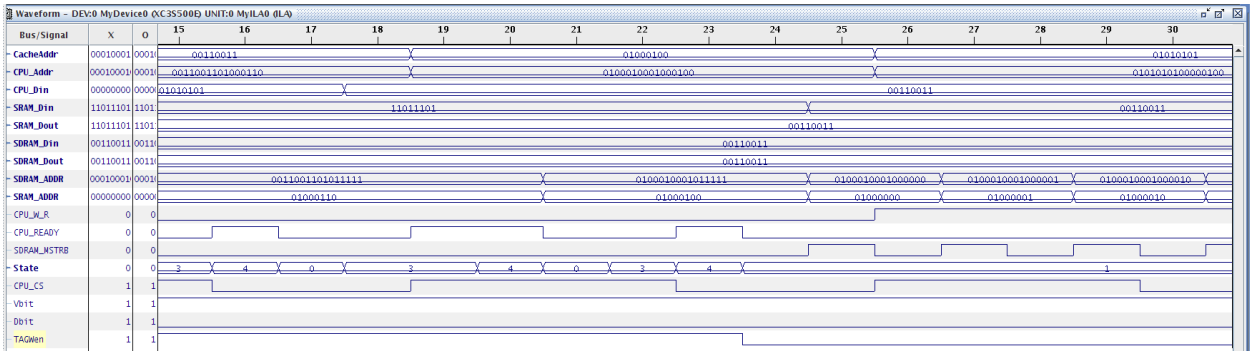

Figure 15: cache controller need to read data, so its states begins as load data from SDRAM, 1

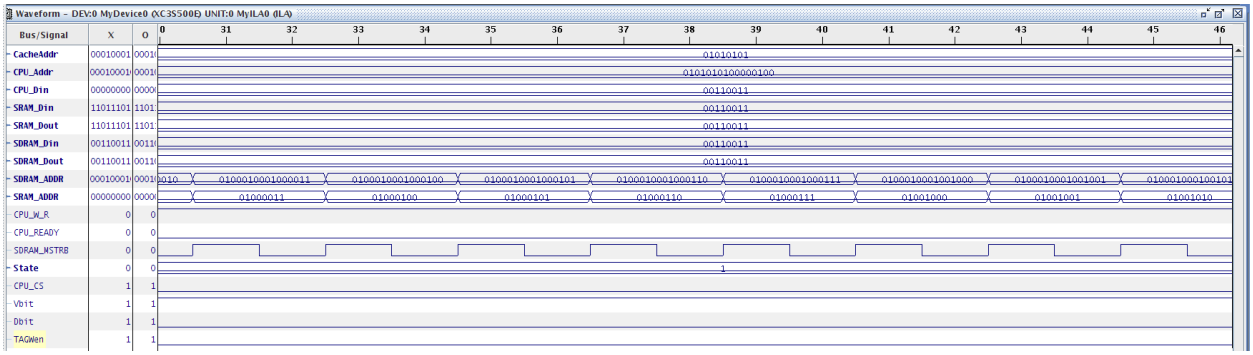

Figure 16: cache being loaded from the SDRAM (continuation of Figure 15)

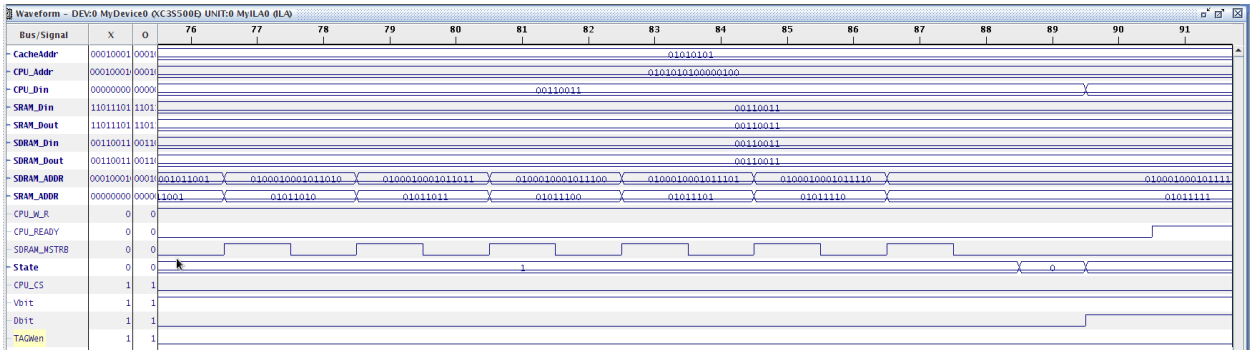

Figure 18: the end of loading date and going into Hit, state 0

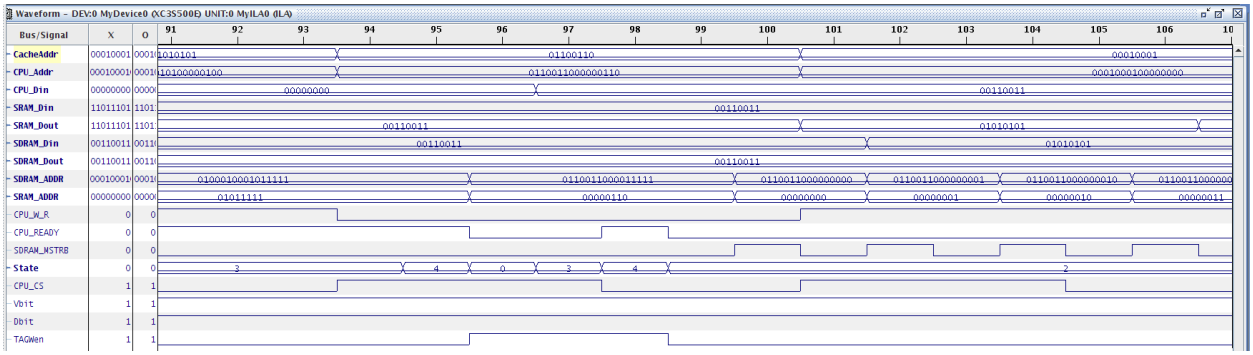

Figure 19: The cache controller needing to write a block to SDRAM memory, so it goes into state 2, write back to memory

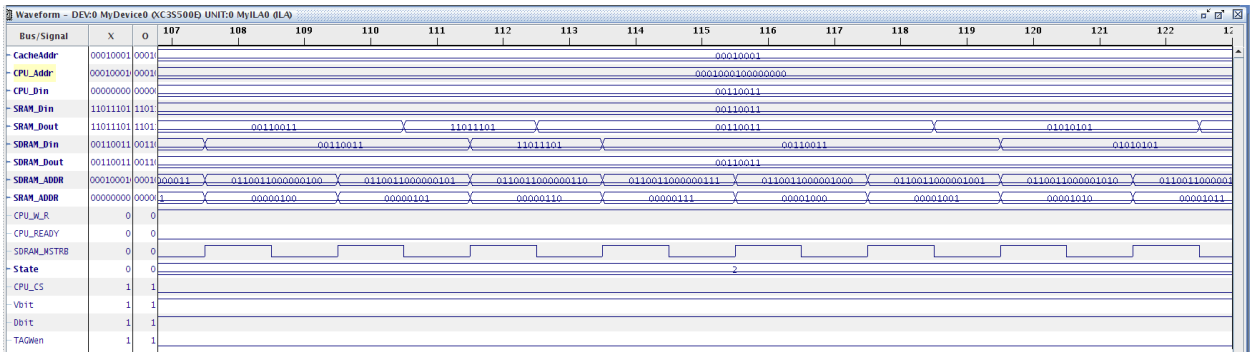

Figure 20: continuation of Figure 19: writing words back to SDRAM

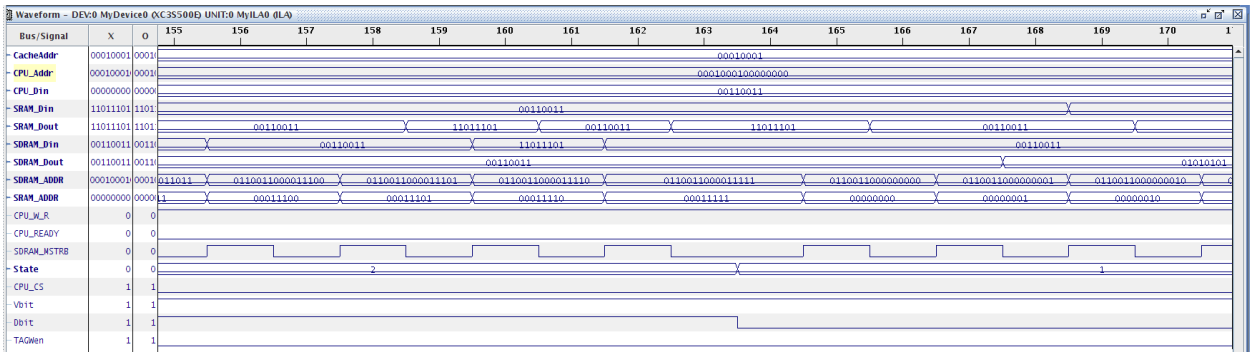

Figure 21: The end of writing back and the beginning of reading from a new block

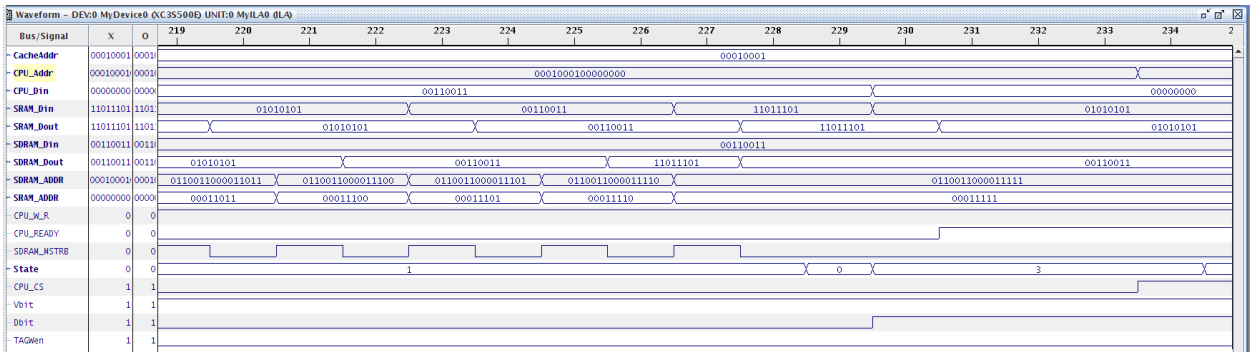

Figure 22: The end of reading from the block of memory and going back to state 0, Hit, then IDL and Ready again

6. Conclusion

From the simulation and analysis results, it can be concluded that the Cache Controller design successfully demonstrates the full functionality of a cache controller in a simplified memory hierarchy system. Through the interaction of the CPU, Cache Controller, local Cache (SRAM), and Main Memory (SDRAM), the project clearly showcases how different levels of memory are communicating and synchronizing to perform efficient data transfers based on the CPU's read and write requests. As it could be seen, the entire operation of the controller, observed through the waveform diagrams and functional simulations, aligns exactly with the expected cache behaviors dictated by the design specification, specifically the four main cases: "Hit-Read", "Hit-Write", "Clean Miss", and "Dirty Miss".

As seen in the simulation timing results, the 'Dirty Miss' operation in the whole takes the longest due to its nature: it first performs a complete block write-back to the main memory before proceeding with the block replacement process. Just the write-back process took approximately 640,000 ns, and as shown in the waveform, 64 clock cycles were used, with every even cycle asserting the SDRAM strobe signal (SDRAM_MSTRB) to allow proper data transfer between the cache and main memory. Each of these even cycles corresponds to one of the 32 words being written back from the cache block to the SDRAM, demonstrating that the 'Dirty Miss' state is effectively handling the write-back operation before moving to the next state.

Immediately following that, the 'Clean Miss' operation occurs, where the cache fetches a new data block from the main memory to replace and overwrite the one just written back. This state displayed a similar transfer pattern to the 'Dirty Miss', lasting about 650,000 ns, and once again involving 32 transfers (two clock cycles per transfer). Finally, after the new block is loaded into the cache, the Hit state executes, completing the CPU's requested operation(As the requested memory space/block is finally in cache memory, it can do its read/write operations). In this case,

a write/read operation occurred very quickly, around 40,000 ns, this proves that the direct cache access is significantly faster, as expected of course.

As a result, the waveforms and timing diagrams validate the Cache Controller behaviour and we understand that the Controllers behave exactly as intended. The state transitions, data transfers, and timing patterns all support the theoretical model of a working cache hierarchy(as taught through class and lab manual). The project demonstrates an accurate and efficient memory management mechanism, where each state performs its specific role before passing control to the next. As it could be seen, these results confirm the successful implementation of modular and synchronized cache operation, accurately reflecting how cache systems function in practical real-world digital architectures.

7. Reference

8. Appendix (code)

```
1
2  --States Machine:
3  --0000 : state0 : Hit/Miss
4  --0001 : state1 : Load from Main Memory
5  --0010 : state2 : Write back to Main Memory
6  --0011 : state3 : IDLE
7  --0100 : state4 : READY
8
9  library IEEE;
10 use IEEE.STD_LOGIC_1164.ALL;
11 use IEEE.NUMERIC_STD.ALL;
12
13 entity CacheController is
14     Port ( clk        : in  STD_LOGIC;
15          ADDR         : out  STD_LOGIC_VECTOR(15 downto 0);
16          cacheAddr    : out  STD_LOGIC_VECTOR(7 downto 0);
17          CS, MEMSTRB, RDY, WR_RD : out  STD_LOGIC;
18          DOUT       : out  STD_LOGIC_VECTOR(7 downto 0);
19          sAddra     : out  STD_LOGIC_VECTOR(7 downto 0);
20          sDina      : out  STD_LOGIC_VECTOR(7 downto 0);
21          sDouta     : out  STD_LOGIC_VECTOR(7 downto 0);
22          sD_Addra   : out  STD_LOGIC_VECTOR(15 downto 0);
23          sD_Dina    : out  STD_LOGIC_VECTOR(7 downto 0);
24          sD_Douta   : out  STD_LOGIC_VECTOR(7 downto 0));
25 end CacheController;
26
27 architecture Behavioral of CacheController is
28 --CPU
29    signal CPU_Dout, CPU_Din    : STD_LOGIC_VECTOR(7 downto 0);
```

```
--CPU
   signal CPU_Dout, CPU_Din   : STD_LOGIC_VECTOR(7 downto 0);
   signal CPU_ADD             : STD_LOGIC_VECTOR (15 downto 0);
   signal CPU_W_R,CPU_CS      : STD_LOGIC;
   signal CPU_RDY             : STD_LOGIC;
   signal cpu_tag             : STD_LOGIC_VECTOR(7 downto 0);
   signal index               : STD_LOGIC_VECTOR(2 downto 0);
   signal offset              : STD_LOGIC_VECTOR(4 downto 0);
   signal Tag_index           : STD_LOGIC_VECTOR(10 downto 0);

--SRAM
   signal Dbit                : STD_LOGIC_VECTOR(7 downto 0):= "00000000";
   signal Vbit                : STD_LOGIC_VECTOR(7 downto 0):= "00000000";
   signal sADD, sDin, sDout   : STD_LOGIC_VECTOR(7 downto 0);
   signal sWen                : STD_LOGIC_VECTOR(0 DOWNTO 0);
   signal TAGWen              : STD_LOGIC := '0';

--SDRAM
   signal SDRAM_Din,SDRAM_Dout   : STD_LOGIC_VECTOR(7 downto 0);
   signal SDRAM_ADD              : STD_LOGIC_VECTOR(15 downto 0);
   signal SDRAM_MSTRB,SDRAM_W_R  : STD_LOGIC;
   signal counter                : integer := 0;
   signal sdoffset               : integer := 0;

   type cachememory is array (7 downto 0) of STD_LOGIC_VECTOR(7 downto 0);
   signal memtag: cachememory := ((others=> (others=>'0')));

   signal control0 : STD_LOGIC_VECTOR(35 downto 0);
   signal ila_data : std_logic_vector(99 downto 0);
   signal trig0    : std_logic_vector(0 TO 0);

   TYPE   state_value IS (state4, state0, state1, state2, state3);
   signal state_current       : state_value ;
   signal state               : STD_LOGIC_VECTOR(3 downto 0);

--Components
   COMPONENT SDRAMController
    Port (
      clk      : in  STD_LOGIC;
      ADDR     : in  STD_LOGIC_VECTOR (15 downto 0);
      DIN      : in  STD_LOGIC_VECTOR (7 downto 0);
      DOUT     : out STD_LOGIC_VECTOR (7 downto 0);
      MEMSTRB  : in  STD_LOGIC;
      WR_RD    : in  STD_LOGIC);

   END COMPONENT;

   COMPONENT SRAM
   PORT (
    clka    : IN STD_LOGIC;
    addra   : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    dina    : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    douta   : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
    wea     : IN STD_LOGIC_VECTOR(0 DOWNTO 0));
   END COMPONENT;

   COMPONENT CPU_gen
   Port (
      clk      : in  STD_LOGIC;
      Address  : out STD_LOGIC_VECTOR (15 downto 0);
      cs       : out STD_LOGIC;
      Dout     : out STD_LOGIC_VECTOR (7 downto 0);
      rst      : in  STD_LOGIC;
      trig     : in  STD_LOGIC;
      wr_rd    : out STD_LOGIC);
   END COMPONENT;


   COMPONENT icon
   PORT (
    CONTROL0 : INOUT STD_LOGIC_VECTOR(35 DOWNTO 0));
   END COMPONENT;


   COMPONENT ila
   PORT (
    CONTROL    : INOUT STD_LOGIC_VECTOR(35 DOWNTO 0);
    CLK        : IN STD_LOGIC;
    DATA       : IN STD_LOGIC_VECTOR(99 DOWNTO 0);
    TRIG0      : IN STD_LOGIC_VECTOR(0 TO 0));
   END COMPONENT;

BEGIN
   myCPU_gen   : CPU_gen         Port Map (clk, CPU_ADD, CPU_CS, CPU_Dout, '0', CPU_RDY,CPU_W_R);
   SDRAM       : SDRAMController Port Map (clk,SDRAM_ADD,SDRAM_Din,SDRAM_Dout,SDRAM_MSTRB, SDRAM_W_R);
```

```
112    SDRAM       : SDRAMController Port Map (clk,SDRAM_ADD,SDRAM_Din,SDRAM_Dout,SDRAM_MSTRB, SDRAM_W_R);
113    mySRAM      : SRAM            Port Map (clk, sADD, sDin, sDout, sWen);
114    myIcon      : icon            Port Map (CONTROL0);
115    myILA       : ila             Port Map (CONTROL0,CLK,ila_data, TRIG0);
116
117 process(clk, CPU_CS)
118    begin
119       if (clk'event AND clk = '1') then
120          if (state_current = state4) then
121             CPU_RDY  <= '0';
122             cpu_tag  <= CPU_ADD(15 downto 8);
123             index    <= CPU_ADD(7  downto 5);
124             offset   <= CPU_ADD(4  downto 0);
125             SDRAM_ADD(15 downto 5)  <= CPU_ADD(15 downto 5);
126             sADD(7 downto 0)        <= CPU_ADD(7 downto 0);
127             sWen <= "0";
128
129             if(Vbit(to_integer(unsigned(index))) = '1'
130                AND memtag(to_integer(unsigned(index))) = cpu_tag) then
131                TAGWen <= '1';
132                state_current  <= state0;
133                state       <= "0000";
134             else
135                TAGWen <= '0';
136                if (Dbit(to_integer(unsigned(index))) = '1'
137                W  AND Vbit(to_integer(unsigned(index))) = '1') then
138                   state_current <= state2;
139                   state <= "0010";
140                else
140                   else
141                      state_current  <= state1;
142                      state <= "0001";
143                end if;
144             end if;
145
146          elsif(state_current = state0) then
147             if (CPU_W_R = '1') then
148                sWen <= "1";
149                Dbit(to_integer(unsigned(index))) <= '1';
150                Vbit(to_integer(unsigned(index))) <= '1';
151                sDin <= CPU_Dout;
152                CPU_Din <= "00000000";
153             else
154                CPU_Din <= sDout;
155             end if;
156
157             state_current <= state3;
158             state <= "0011";
159
160          elsif(state_current = state1) then
161             if (counter = 64) then
162                   counter <= 0;
163                   Vbit(to_integer(unsigned(index))) <= '1';
164                   memtag(to_integer(unsigned(index))) <= cpu_tag;
165                   sdoffset <= 0;
166                   state_current <= state0;
167                   state <= "0000";
168                else
169                   if (counter mod 2 = 1) then
170                      SDRAM_MSTRB <= '0';
171                   else
172                      SDRAM_ADD(4 downto 0) <= STD_LOGIC_VECTOR(to_unsigned(sdoffset, offset'length));
173                      SDRAM_W_R <= '0';
174                      SDRAM_MSTRB <= '1';
175                      sADD(7 downto 5) <= index;
176                      sADD(4 downto 0) <= STD_LOGIC_VECTOR(to_unsigned(sdoffset, offset'length));
177                      sDin <= SDRAM_Dout;
178                      sWen <= "1";
179                      sdoffset <= sdoffset + 1;
180                   end if;
181                   counter <= counter + 1;
182                end if;
183
184          elsif(state_current = state2) then
185             if (counter = 64) then
186                   counter <= 0;
187                   Dbit(to_integer(unsigned(index))) <= '0';
188                   sdoffset <= 0;
189                   state_current <= state1;
190                   state <= "0001";
191                else
192                   if (counter mod 2 = 1) then
193                      SDRAM_MSTRB <= '0';
194                   else
195                      SDRAM_ADD(4 downto 0) <= STD_LOGIC_VECTOR(to_unsigned(sdoffset, offset'length));
196                      SDRAM_W_R <= '1';
```

```
                    sADD(7 downto 5) <= index;
                    sADD(4 downto 0) <= STD_LOGIC_VECTOR(to_unsigned(sdoffset, offset'length));
                    sWen <= "0";
                    SDRAM_Din <= sDout;
                    SDRAM_MSTRB <= '1';
                    sdoffset <= sdoffset + 1;
                 end if;
                 counter <= counter + 1;
              end if;

           elsif(state_current = state3) then
              CPU_RDY <= '1';
              if (CPU_CS = '1') then
                 state_current <= state4;
                 state <= "0100";
              end if;
           end if;
     end if;
end process;

--Mapping
   ADDR     <= CPU_ADD;
   CS       <= CPU_CS;
   DOUT     <= CPU_Din;
   MEMSTRB  <= SDRAM_MSTRB;
   RDY      <= CPU_RDY;
   WR_RD    <= CPU_W_R;

   sAddra <= sADD;
   sDina  <= sDin;
   sDouta <= sDout;

   sD_Addra <= SDRAM_ADD;
   sD_Dina  <= SDRAM_Din;
   sD_Douta <= SDRAM_Dout;
   cacheAddr <= CPU_ADD(15 downto 8);

   ila_data(15 downto 0) <= CPU_ADD;
   ila_data(16) <= CPU_W_R;
   ila_data(17) <= CPU_RDY;
   ila_data(18) <= SDRAM_MSTRB;
   ila_data(26 downto 19) <= CPU_Din;
   ila_data(30 downto 27) <= state;
   ila_data(31) <= CPU_CS;
   ila_data(32) <= Vbit(to_integer(unsigned(index)));
   ila_data(33) <= Dbit(to_integer(unsigned(index)));
   ila_data(34) <= TAGWen;
   ila_data(42 downto 35) <= sADD;
   ila_data(50 downto 43) <= sDin;
   ila_data(58 downto 51) <= sDout;
   ila_data(74 downto 59) <= SDRAM_ADD;
   ila_data(82 downto 75) <= SDRAM_Din;
   ila_data(90 downto 83) <= SDRAM_Dout;
   ila_data(98 downto 91) <= CPU_ADD(15 downto 8);

end Behavioral;
```

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity SDRAMController is
6      Port (
7             clk       : in STD_LOGIC;
8             ADDR      : in  STD_LOGIC_VECTOR (15 downto 0);
9             WR_RD     : in  STD_LOGIC;
10            MEMSTRB      : in  STD_LOGIC;
11            DIN       : in  STD_LOGIC_VECTOR (7 downto 0);
12            DOUT      : out  STD_LOGIC_VECTOR (7 downto 0));
13 end SDRAMController;
14
15 architecture Behavioral of SDRAMController is
16     type ramemory is array (7 downto 0, 31 downto 0) of std_logic_vector(7 downto 0);
17     signal RAM_SIG: ramemory;
18
19    signal counter : integer := 0;
20 begin
21
22 process (CLK)
23     begin
24         if CLK'event and CLK = '1' then
25
26             if counter = 0 then
27                for I in 0 to 7 loop
28                   for J in 0 to 31 loop
29                      RAM_SIG(i,j) <= "11110000";
30                   end loop;
31                end loop;
32                counter <= 1;
33             end if;
34
35             if MEMSTRB = '1' then
36                if WR_RD = '1' then
37                   RAM_SIG(to_integer(unsigned(ADDR(7 downto 5))),to_integer(unsigned(ADDR(4 downto 0)))) <= DIN;
38                ELSE
39                   DOUT <= RAM_SIG(to_integer(unsigned(ADDR(7 downto 5))),to_integer(unsigned(ADDR(4 downto 0))));
40                end if;
41             end if;
42         end if;
43     end process;
44
45
46 end Behavioral;
```